

TeRed: Normal Behavior-Based Efficient Provenance Graph Reduction for Large-Scale Attack Forensics

Xiaoxiang Li[✉], Xinyu Jiang, Hai Wan[✉], and Xibin Zhao[✉], *Senior Member, IEEE*

Abstract—System intrusions, particularly Advanced Persistent Threats (APTs), pose significant threats to enterprises and organizations. Provenance graph-based attack detection and investigation methods are crucial for defending against these intrusions. To detect various attacks, security systems collect comprehensive operating system event data, resulting in massive provenance graphs that increase storage costs and complicate analysis and querying. Efficiently optimizing these provenance graphs has thus become a core issue. However, existing data reduction methods often mistakenly delete critical security information, significantly impacting attack detection and investigation. This paper introduces TeRed, a novel method for reducing provenance graphs based on normal behavior patterns. Our approach employs unit tests to learn the system's normal behavior patterns, which are then used to streamline the provenance graph. Experiments on five datasets show that our method reduces the provenance graph while preserving all attack-related information. Importantly, it does not compromise attack detection and investigation, showcasing significant advantages over other data reduction techniques.

Index Terms—Data reduction, provenance graph, template mining, intrusion detection, attack investigation.

I. INTRODUCTION

CURRENTLY, with the increasing complexity of cyber-attack methods, system intrusions represented by Advanced Persistent Threats (APTs) pose a significant threat to various enterprises and organizations. These threats can not only lead to the leakage of sensitive data but also severely impact the operations of enterprises. Intrusion detection and attack investigation tools, which can promptly detect abnormal

behaviors and trace the attacker's activity path, are considered important means of defending against system intrusions. Presently, these tools primarily rely on provenance graphs, which depict the information flow between system entities as directed graphs, effectively illustrating the system's execution history. Provenance graphs not only visually reconstruct attack paths [1] but also offer rich contextual information, aiding in the differentiation between benign and malicious events [2].

In the real world, to detect covert attacks that have been lurking for a long time, security systems comprehensively collect operating system event data. The scale of this event data is enormous, with a single host generating over 1GB of data per day [3]. Consequently, the provenance graphs constructed from the massive system event data are also immense in scale. Large-scale provenance graphs not only increase storage costs but also make analysis difficult. Therefore, efficiently optimizing provenance graphs has become a core issue.

Data reduction has emerged as a key optimization technique. The primary objective of data reduction is to minimize data size while maintaining usability. During processing, the system retains only the streamlined data, with the original data either discarded or cold-stored. Therefore, the reduced data must first support attack detection and investigation, with achieving a high reduction ratio being secondary. This means the reduction process must preserve all malicious behavior data, ensuring complete attack-related information is retained. Additionally, the reduced data should be directly usable without restoring.

There has been a significant amount of work on reducing provenance graphs, but none of these methods can fully meet the aforementioned requirements. While lossless compression methods do not lose any data, the compressed data needs to be decompressed before use, which increases the runtime system load. Lossy reduction methods optimize and remove redundant events irrelevant to the investigation process. However, these methods cannot achieve high reduction ratios without negatively impacting downstream tasks. Experiments by Inam et al. on the DARPA-TC dataset [4] show that only reduction algorithms with reduction ratios below 10% avoid adversely affecting intrusion detection tasks. High reduction ratio algorithms lead to a significant number of false positives and false negatives in subsequent intrusion detection algorithms. For instance, the SDPR [5] algorithm achieves a high reduction ratio of 92.45%, but it causes the precision of the DeepLog [6] algorithm to drop from 52.94% to 4.88% and the recall to

Received 20 January 2025; revised 16 July 2025; accepted 8 August 2025. Date of publication 21 August 2025; date of current version 15 September 2025. This work was supported in part by Guangdong Science and Technology Program under Grant 2024B0101030002, in part by NSFC under Grant 6212780016, in part by the Ministry of Industry and Information Technology of China, in part by the National Key Research and Development Program of China under Grant 2023YFB3307500, and in part by the Science and Technology Innovation Project of Hunan Province under Grant 2023RC4014. The associate editor coordinating the review of this article and approving it for publication was Dr. Weizhi Meng. (*Corresponding author: Xibin Zhao.*)

The authors are with Beijing National Research Center for Information Science and Technology (BNRist), Key Laboratory for Information System Security, Ministry of Education (KLIS), School of Software, Tsinghua University, Beijing 100084, China (e-mail: xx-li24@mails.tsinghua.edu.cn; jiang-xy23@mails.tsinghua.edu.cn; wanhai@tsinghua.edu.cn; zxb@tsinghua.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TIFS.2025.3601381>, provided by the authors.

Digital Object Identifier 10.1109/TIFS.2025.3601381

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: NATIONAL INSTITUTE OF TECHNOLOGY CALICUT. Downloaded on April 01, 2026 at 02:02:54 UTC from IEEE Xplore. Restrictions apply.

drop from 87.1% to 14.29%. Such a trade-off is unacceptable to security experts.

The negative impacts of reduction on downstream tasks mainly come from two aspects: incorrect reduction of anomalous behaviors and disruption of the graph structure. Intrusion detection relies on identifying anomalous behaviors, so the reduction process must retain data related to these behaviors. Indiscriminate reduction can result in the loss of crucial data, diminishing detection effectiveness. Attack investigation depends on reconstructing the attack path through both forward and backward tracing, so maintaining the graph structure is essential. Disrupting the graph structure can sever dependencies between entities, leading to truncated tracing paths or erroneous dependencies.

Hence, the reduction method must retain anomalous data and preserve graph integrity. Identifying anomalous behavior is challenging and lies at the core of most intrusion-detection algorithms, whereas normal behavior is comparatively easier to recognize. By learning normal-activity patterns from benign execution traces, locating them in the provenance graph, and pruning the corresponding nodes and edges, we can avoid erroneously removing anomalies. However, this normal-behavior-based reduction method faces several challenges. First, ensuring broad applicability demands fine-grained patterns, which require numerous provenance graphs containing single normal behaviors as training data. Second, accurately extracting those patterns from complex, noisy provenance graphs calls for effective noise elimination while keeping relevant information. Third, during graph reduction the system must match patterns precisely to prevent accidental trimming of suspicious data and to maintain the dependencies that guarantee forward and backward reachability. Finally, the integrity of the overall graph structure must be safeguarded so that subsequent analyses remain reliable.

This paper presents TeRed, a provenance graph reduction method that effectively addresses the aforementioned challenges. TeRed learns the system's normal behavior patterns from unit test data, identifies the corresponding normal behaviors in the provenance graph, and optimizes and trims them. This allows for effective reduction of the provenance graph without negatively impacting attack detection and investigation. To our knowledge, this is the first data reduction method that performs reduction on provenance graphs based on normal behavior patterns.

TeRed consists of two phases: the learning phase and the reduction phase. In the learning phase, the method starts with unit test cases of the program and obtains provenance graphs corresponding to different normal behaviors by executing these unit tests. These provenance graphs meticulously reflect various behaviors and operations of the program under normal operating conditions. Next, the method uses a learner based on the gSpan algorithm to learn from the provenance graph data and extract graph patterns that represent normal behavior patterns. Through this algorithm, we effectively eliminate system noise from the provenance graphs and extract graph patterns that represent normal behavior patterns. These patterns serve as templates for identifying and understanding normal behaviors. During the reduction phase, the method uses a subgraph

isomorphism algorithm to identify subgraph structures in the provenance graph to be reduced that are isomorphic to the templates. These identified subgraph structures represent the normal behaviors indicated by the templates and are thus targets for reduction. The method optimizes these subgraph parts by trimming the internal edges and nodes of the subgraph structures and reconstructing the original dependency relationships, ensuring that reduction is achieved while preserving forward and backward dependency relationships.

We implemented a prototype system of TeRed and evaluated the method on datasets, comparing it with other advanced methods in the field. The datasets include three popular vulnerabilities from the publicly available Vulhub and the DARPA-TC project dataset. The evaluation results show that TeRed achieves a high reduction ratio, not lower than most other existing methods. More importantly, TeRed does not negatively impact the accuracy of intrusion detection tasks, ensuring the precision of detection. Additionally, the reduced data retains the dependency relationships outside the reduced regions, effectively supporting attack detection and investigation. The TeRed prototype is publicly available at <https://github.com/THSS-GAD/TeRed>.

In summary, the contributions of this paper are as follows:

- We propose TeRed, a provenance graph reduction framework driven by normal-behavior patterns. By reducing only those substructures that match these patterns, TeRed achieves substantial data reduction without harming downstream tasks. To the best of our knowledge, it is the first approach to apply normal-behavior patterns to provenance graph reduction.
- We introduce a method for extracting normal behavior patterns based on unit tests. This method accurately extracts various normal behavior patterns from programs and continuously updates them to adapt to system updates and changes.
- We evaluate TeRed on five datasets, and the results show that it can achieve a high reduction ratio while maintaining intrusion-detection accuracy and preserving attack-related information.

II. BACKGROUND AND MOTIVATION

A. Provenance-Based Attack Detection and Investigation

Data provenance represents the origin of events within a system and the trajectory leading to the current state, recording the data flow between system entities. Data provenance can be represented as a directed graph, known as a provenance graph. In a provenance graph, a node represents a system entity, including processes, files, sockets, and hosts, each with a unique identifier and some other attributes related to the specific entity type. Directed edges represent events between two entities, including read, write, clone, and other events. An edge contains at least four elements: a unique identifier, the unique identifiers of the source and target nodes, and a timestamp. Provenance graphs can effectively model the relationships between entities within a system, providing security personnel with a highly visual representation of event-related entities and their relationships. Well-constructed provenance graphs can

effectively display high-level semantic information, helping security personnel to detect threats and correlate attacks, and thus have been widely used in numerous attack detection and provenance tracking efforts.

Intrusion detection [2], [6], [7], [8], [9], [10], [11] aims to monitor and detect malicious activities within a system by analyzing system events. Many current methods use provenance graphs for this purpose. Unlike other detection techniques, provenance-based methods analyze not only system entities and their attributes but also the causal relationships and information flow between entities. This enables better detection of unknown attacks, such as zero-day attacks. These methods operate on the observation that attack behaviors are often low-probability events. They build models by learning from a large amount of benign behavior data and use these models to identify low-probability behaviors in the provenance graph. By analyzing these behaviors, they can detect attacks and identify related nodes and edges. Security personnel can then conduct further analysis and investigation based on the detected attacks.

To support these detection methods, data reduction must preserve the frequency information of various behaviors to avoid misclassifying normal behaviors as low-probability. Additionally, it must retain information related to attack behaviors to prevent misidentifying attacks as normal behaviors.

Attack investigation aims to infer the chain of events leading to an intrusion and assess the potential damage to the system [12], [13], [14], [15], [16], [17]. This process is divided into two parts: backward tracing and forward tracing. Backward tracing starts from the alert event and traces the attack path in reverse to identify the origin of the attack, thereby understanding the attack process and methods for vulnerability remediation. Forward tracing also starts from the alert event, tracking its possible behavior paths and impacts to identify affected system entities, preventing further attacks and reducing system damage. Security personnel use the dependency relationships between entities in the provenance graph to conduct attack investigations, reconstruct the complete attack scenario, quickly understand the intrusion, and design timely responses.

To avoid negatively impacting the attack investigation process, the reduction process must preserve the dependency relationships between entities in the provenance graph. This ensures that the provenance paths are not cut off or incorrectly linked.

B. Data Reduction

To achieve comprehensive security protection, it is necessary to collect and store system events from each host, which imposes significant pressure on data storage. On one hand, the volume of system events generated by current computer systems during operation is enormous, with each host capable of producing over 1GB of system event data per day [3]. On the other hand, because attacks often lurk in the system for a long time, it is necessary to retain system event data for extended periods. The massive volume of data not only puts immense pressure on storage but also makes attack detection and investigation akin to finding a needle in a haystack. Security personnel have to search for potential attack

behaviors in a provenance graph containing tens of thousands of nodes and edges, which leads to delayed responses to attack behaviors.

Currently, there has been a significant amount of work attempting to solve this problem by pruning and optimizing unnecessary nodes and edges in the provenance graph. Some methods prune events that are meaningless for attack detection and investigation. For example, LogGC [18] identifies and deletes temporary files, while NodeMerge [19] identifies and deletes fixed file combinations loaded during application startup. Other methods (CPR [20], FDP and SDPR [5]) delete a large number of file I/O-related events while retaining events related to information flow, thereby reducing the log size. Additionally, some methods (LogApprox [21], PCAR [20]) perform extensive deletion optimizations, retaining only data related to causal relationships. These methods can reduce data size to varying degrees. Unfortunately, the high reduction ratio of existing work come at the cost of significant damage to attack-related information.

Research by Inam et al. [4] shows that LogGC and NodeMerge can reduce data volume by about 10% without causing a decline in intrusion detection metrics. CPR and PCAR reduce data volume by about 30%, but at the cost of some attack data loss. Methods like LogApprox, FD, and SD can significantly reduce data volume but also cause a noticeable decline in intrusion detection metrics. The SD method achieves over 90% data reduction but causes the detection accuracy of the DeepLog method to drop from 52.94% to 4.88% and recall rate from 87.10% to 14.29%. These methods indiscriminately reduce all areas of the graph, leading to erroneous pruning of attack-related information and loss of frequency information, thereby causing intrusion detection methods to fail. Specifically, LogGC and NodeMerge reduce only a limited subset of runtime-observed benign activity, leaving the structural and statistical cues required for intrusion detection substantially intact. By contrast, CPR and PCAR aggressively collapse the structural neighbourhoods surrounding attack nodes, resulting in the inadvertent removal of attack-critical evidence. LogApprox coalesces edges that encode semantically similar actions, whereas FD and SD eliminate the majority of edges with equivalent reachability semantics; in both cases, the consequent erasure of frequency information undermines intrusion-detection techniques whose efficacy depends on event frequency. The significant decline in metrics renders these high-reduction data reduction methods ineffective in supporting subsequent attack detection and investigation.

It is evident that existing data reduction methods cannot achieve high reduction ratios while maintaining support for downstream tasks. Therefore, this paper aims to propose a new data reduction method to address the issue of negative impacts on downstream tasks caused by extensive reduction.

C. Our Method

To ensure that the reduction process does not negatively impact downstream tasks, it is essential to focus on reducing normal behavior data within the system. Normal behavior data, generated during non-malicious operations, is unrelated

to attack behaviors and can thus be reduced. Identifying these normal behaviors in the provenance graph is a key step. To achieve this, a set of normal behavior patterns must be obtained in advance, serving as references for identifying normal behaviors during the reduction process.

The set of normal behavior patterns of the system consists of a large number of different behavior patterns, each representing a specific type of program operation, such as downloading files from the network, writing and saving files, etc. These behavior patterns are defined based on the normal operation of the system and reflect the behaviors and operations of the system when it is free from attack.

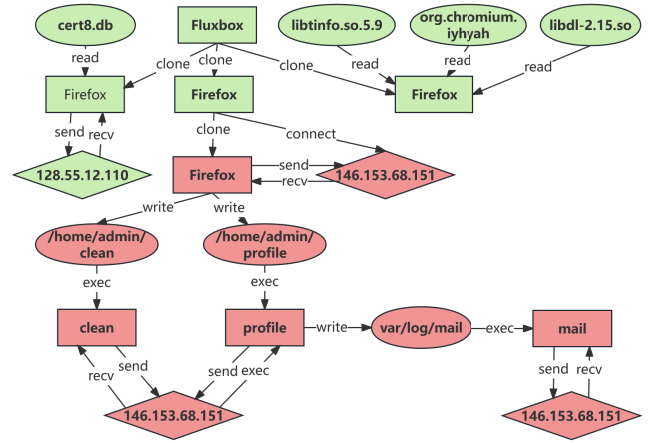
However, due to the complexity and diversity of the system, obtaining a comprehensive set of normal behavior patterns is very challenging. Extracting data corresponding to individual behaviors from complex system behavior data is difficult because system behaviors often involve multiple entities and operations with intricate interactions and dependencies. Transforming this complex data into understandable behavior patterns is a significant challenge. Besides, the highly diverse nature of system behaviors makes it hard for any dataset to encompass all normal behaviors, meaning the obtained behavior patterns may not cover every normal behavior. Therefore, it is essential to continuously update and expand the set of behavior patterns to adapt to the evolving system behaviors.

To address these challenges, we focus on unit testing, a crucial part of the software development process that verifies the correct operation of individual modules or functions. A comprehensive set of unit tests can cover most normal behaviors of a program, allowing us to generate normal behavior data and extract patterns from it. By executing a series of unit tests across various functions, we record system behavior data during testing. This data reflects the program's normal operations and can be considered normal behavior data. We then analyze and process this data to extract behavior patterns. This approach allows us to extract patterns that encompass the majority of the program's normal behaviors. Additionally, leveraging the automation features of unit testing ensures that the pattern extraction process remains continuous and adaptable to program updates and changes. This method effectively addresses the outlined challenges.

Based on the above idea, we have designed and implemented a reduction method based on normal behavior patterns.

1) *Generating Normal Behavior Data From Unit Test Cases:* First, we execute a series of unit test cases and collect complete event data to construct normal behavior provenance graphs. Each test case is executed multiple times to obtain multiple provenance graphs representing the same behavior, supporting the extraction of normal behavior patterns.

2) *Extracting Normal Behavior Patterns:* Next, we use data mining algorithms to extract normal behavior patterns from the data. Specifically, a frequent itemset-based algorithm extracts the maximum common subgraph from multiple provenance graphs representing the same behavior. This subgraph, called the template subgraph, represents the behavior pattern. By mining the provenance graphs of different behaviors, we obtain a series of template subgraphs for the subsequent reduction process.



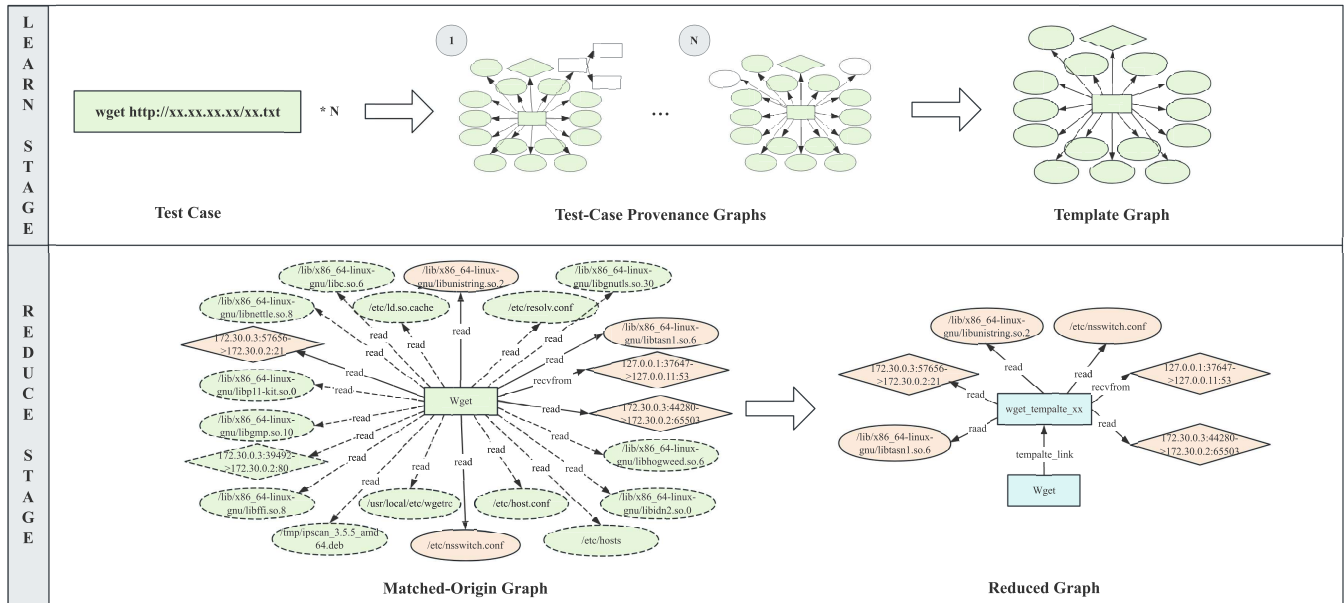


Fig. 2. An example of TeRed: the method runs the Wget network download test cases multiple times to obtain the corresponding provenance graph set. Subsequently, template subgraphs representing the Wget network download behavior are extracted from this set. These templates are then matched against the provenance graph to be reduced, identifying the Wget network download behaviors within the graph for reduction.

kernel-level attacks do not occur and that events generated during system operation are recorded completely and accurately. Furthermore, we assume that the data is not tampered with, and the provenance graph is complete and securely stored. These assumptions can be ensured by existing security technologies [22], [23], [24], [25].

IV. DESIGN

A. Overview

Based on the above ideas, this paper proposes a template-based reduction method aimed at effectively extracting, matching, and reducing normal behavior data by identifying and utilizing common templates. This method is divided into three main stages: the template learning phase, the template matching phase, and the template-based reduction phase.

B. Template Learning

The objective of template learning is to extract templates representing normal behavior patterns from extensive normal behavior data. This involves filtering out randomness and irrelevant content to obtain stable, accurate, and reliable patterns, forming the basis for subsequent template matching and reduction. Template learning comprises two main processes: generating normal data and mining subgraph templates.

1) *Normal Data Generating*: In the template learning phase, generating high-quality normal behavior data is crucial. This data must meet two essential criteria: it should be free of attack activities and correspond specifically to individual behaviors. Ensuring the dataset is free from malicious actions prevents misleading results during template learning and matching, guaranteeing that the extracted templates represent normal system behavior. Additionally, the data should

reflect specific software behaviors, excluding other information to streamline template extraction. This ensures the templates accurately represent individual behaviors and enhances precision during the reduction phase when matching against behaviors within the provenance graph.

To meet the above requirements, we use normal test cases from software unit testing to generate the required data. The primary purpose of designing unit test cases is to verify the correctness and robustness of software functionality, and they do not contain any attack behaviors. Additionally, unit test cases focus on specific functional points of the software, executing one specific action at a time. For example, a unit test might call a function and verify whether the function's output meets the expectations. In this way, the data collected during each test execution can represent a specific behavior.

Furthermore, the software development process provides a rich and comprehensive set of test cases that cover different functionalities of the software. By using these diverse test cases, we can significantly enhance the coverage of the generated data. The high coverage test cases provide data encompassing a wide range of system operations, thereby increasing the number and coverage of the normal behavior templates extracted, ultimately enhancing the overall reduction ratio of the system.

Consequently, we select verified unit test cases with broad coverage from the software development test library. A test case is retained only if (i) it triggers exactly one atomic operation, (ii) it contains no boundary-condition or fault-injection logic, and (iii) its inclusion raises statement/branch coverage. Then, we execute the chosen unit test cases in bulk using automated testing tools. Finally, we collect the extensive logs produced by executing the unit tests, followed by data cleansing and format transformation to construct individual

provenance graphs. These steps allow us to generate a set of high-quality normal data that is free of attacks and where each provenance graph corresponds to a single behavior, thereby establishing a solid foundation for subsequent template subgraph mining.

2) *Template Subgraph Mining*: Each subgraph template represents a general pattern of normal behavior. We attempt to utilize the generated normal behavior data to mine generalized provenance graph representations of various normal behaviors.

Program execution involves inherent randomness. Firstly, fields such as the PID during program execution are random, leading to variations in the provenance graph information with each execution. Secondly, during the execution of test cases, the program may interact with entities unrelated to the test case. These random and irrelevant interactions can affect the resulting provenance graph. Therefore, we cannot directly use the graphs obtained from test case executions as subgraph templates representing normal behavior patterns.

To obtain template subgraphs, we need to process the provenance graphs in a way that preserves their basic topological structure, node types, edge types, and other key information while filtering out fields influenced by randomness and irrelevant interactions. The specific steps are as follows: Firstly, execute the same test case multiple times to generate multiple provenance graphs. Then, analyze these provenance graphs to extract their common parts as template subgraphs. Essentially, this is a frequent subgraph mining process.

Definition 1 (Graph Dataset): Graph dataset $D = \{G_1, G_2, \dots, G_n\}$ is a set of provenance graphs, where each graph G_i is a provenance graph defined as $G_i = (V_i, E_i)$.

Definition 2 (Frequent Subgraph): Let τ be the support threshold. A subgraph s is considered frequent if and only if $\text{support}(s, D) \geq \tau$. The support of frequent subgraph s is defined as

$$\text{support}(s, D) = |\{G_i \in D : \text{count}(s, G_i) > 0\}|$$

where $\text{count}(s, G_i)$ denotes the number of times s appears as a subgraph in G_i .

Frequent subgraphs can represent patterns that consistently appear across multiple executions, serving as template subgraphs for corresponding behaviors. In this paper, we use gSpan [26] as the foundational algorithm for frequent subgraph mining, and the algorithm framework is illustrated in Algorithm 1. The core idea of gSpan is to discover frequent subgraphs through DFS (Depth First Search), efficiently handling large-scale graph databases. The first step of the algorithm is to remove infrequent nodes and edges from the dataset and place the remaining edges in the output set according to DFS coding order. The second step iteratively invokes the SubMining growth algorithm on each frequent edge graph, continuously finding frequent subgraphs with more edges. The third step removes edges from the dataset that are identical to the mined edges, further reducing the scale. Finally, the algorithm checks for support; when the number of graphs in the dataset falls below the support threshold, the algorithm terminates.

The iterative module SubMining, as shown in Algorithm 2, expands the given subgraph to find all subsequent frequent

Algorithm 1 Frequent Subgraph Mining

Require: Graph database D , Minimum support τ

Ensure: Mining results S

- 1: Sort points and edges in D by label frequency
 - 2: Remove low frequency points and edges
 - 3: Re-mark the remaining points and edges of D
 - 4: $S^1 \leftarrow$ graphs in D that contain only one edge
 - 5: Arranges the edges in S^1 in DFS dictionary order
 - 6: $S \leftarrow S^1$
 - 7: **for** e in S^1 **do**
 - 8: $s \leftarrow$ graphs in D that contain e
 - 9: $\text{SubMining}(D, s, S, \tau)$
 - 10: $D \leftarrow D - e$
 - 11: **if** $|D| < \tau$ **then**
 - 12: **break**
 - 13: **end if**
 - 14: **end for**
-

Algorithm 2 SubMining

Require: Graph database D , Input subgraph s , Mining results S , Minimum support τ

Ensure: Mining results S

- 1: **if** $s \neq \min(s)$ **then**
 - 2: **return**
 - 3: **end if**
 - 4: $S \leftarrow S \cup \{s\}$
 - 5: Generate all s' potential children with one edge growth in D
 - 6: **for each** s' child c **do**
 - 7: **if** $\text{support}(c, D) \geq \tau$ **then**
 - 8: $s \leftarrow c$
 - 9: $\text{SubMining}(D, s, S, \tau)$
 - 10: **end if**
 - 11: **end for**
-

graphs. Each recursion extends the current subgraph in DFS tree order, attempts to add a new edge to it, and removes duplicate graphs. If the expanded code corresponds to the minimal code of a graph, it gets added to the result set. All generated graphs that meet the requirements enter the next round of recursive calls. This expansion process allows the algorithm to maximize the size of the frequent subgraphs, thus enabling the inclusion of larger areas in subsequent reduction processes and ultimately improving the reduction rate.

Through the template subgraph mining process, we can effectively filter out noise from program executions (which is manifested in the provenance graph as infrequent nodes and edges), resulting in template subgraphs that represent normal behaviors. This collection of subgraphs, which eliminates random fluctuations and irrelevant interactions, lays a solid foundation for subsequent template matching and data reduction stages.

C. Template Matching

Data reduction should focus on data related to normal behavior. Therefore, when reducing the provenance graph, the first step is to identify the reducible parts. If regions in

the provenance graph share the same structure and similar attributes as the template subgraph, it indicates that the behavior represented aligns with the template's behavior pattern. Consequently, the challenge becomes identifying the template subgraph in the provenance graph, which is essentially a subgraph matching problem.

Definition 3 (Subgraph Matching): Given two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, the subgraph matching problem requires determining whether there exists a mapping $f: V_2 \rightarrow V_1$ such that:

- **One-to-One Mapping:** f is injective, that is, $\forall v_i, v_j \in V_2, v_i \neq v_j \Rightarrow f(v_i) \neq f(v_j)$.
- **Adjacency Relationship Preservation:** $\forall v_i, v_j \in V_2, (v_i, v_j) \in E_2 \Rightarrow (f(v_i), f(v_j)) \in E_1$.
- **Attribute Matching:** $\forall v \in V_2, \text{label}(v) = \text{label}(f(v))$, and $\forall v_i, v_j \in V_2, \text{label}(v_i, v_j) = \text{label}(f(v_i), f(v_j))$.

If the above conditions hold, then graph G_2 is said to be a subgraph of graph G_1 .

During the template matching process, it is essential to ensure that the provenance graph regions are consistent with the subgraph in structure and attributes, satisfying the last two conditions of Definition 3, known as **feasibility conditions**. Our algorithm requires that attribute matching not only aligns the entity types of corresponding nodes but also ensures that the system calls for the edges are identical. This guarantees that the behaviors represented by the identified subgraph match those of the template.

We employ the VF2 algorithm [27] for template matching, which is a highly efficient and precise method designed to address subgraph matching problems. The algorithm is based on a recursive approach, dividing the matching process into forward and backtracking phases. In the forward phase, the algorithm attempts to match the nodes in the two graphs and verifies if the feasibility conditions are met. If successful, it proceeds further; if not, it backtracks to try other nodes until no viable nodes remain for further matching.

Algorithm 3 VF2_Match

Require: Graph G_1, G_2 , Intermediate state s (initial state s_0 satisfies $M(s_0) = \emptyset$)
Ensure: Matching result $M(s)$

```

1: if  $M(s)$  contains all the nodes of  $G_2$  then
2:   return  $M(s)$ 
3: else
4:   Evaluates the set  $P(s)$  of the candidate pairs contained
   in  $M(s)$ 
5:   for each  $(p, q)$  in  $P(s)$  do
6:     if Incorporating  $(p, q)$  into  $M(s)$  satisfies the feasi-
     bility conditions then
7:        $s' \leftarrow s \cup (p, q)$ 
8:       VF2_Match ( $G_1, G_2, s'$ )
9:     end if
10:  end for
11:  Save data structure
12: end if
```

The specific algorithmic flow is shown in Algorithm 3. In the algorithm, the state s represents a condition in which a

partial match has been found during the matching process. The mapping $M(s)$ under state s indicates a mapping relationship between the subgraphs $G_1(s)$ and $G_2(s)$ of graphs G_1 and G_2 . The transition from state s to the next state s' involves the process of adding a new pair of nodes (p, q) to the matching set. For each state s the algorithm calculates a set of candidate node pairs $P(s)$ that can be added to the state. For each pair (p, q) in this set, if including it satisfies the feasibility conditions, the pair is added to state s , resulting in a new state s' . The algorithm then iteratively invokes the matching function for the new state s' until the entire matching process is complete.

The two input graphs for the algorithm are the provenance graph to be reduced and the template subgraph from the template library. The algorithm adds the application name attribute to specific nodes in the template subgraph (the application node executing the test case) and designates them as the initial matching nodes. This operation effectively reduces the number of invalid matches and significantly improves algorithm efficiency, especially when there are many template subgraphs in the template library. During the matching process, the provenance graph to be reduced is matched with each template subgraph in the library. Successful matches, including both the matched templates and their positions, are recorded and will define the region for the subsequent reduction phase.

D. Template-Based Reduction

To support causal analysis, after obtaining the template matching results, it is necessary to reduce the provenance graph while minimizing the number of nodes and edges in the regions to be reduced, without losing the original information. Each region to be reduced corresponds to a template subgraph, which represents a normal system behavior. Therefore, the information within the region can be viewed as an entity unrelated to attack information, allowing us to reduce this region. Specifically, our reduction algorithm first identifies the nodes and edges within the region to be reduced and determines the corresponding template subgraph for that region. Next, the algorithm deletes these original nodes and edges, using a new, smaller set of nodes and edges to represent the behavior of the region and associate it with other nodes.

Based on this approach, the algorithm turns the region into two nodes and one edge. The first node represents the application node corresponding to the template, while the second node is a new node generated based on the template information, representing all nodes within the region to be reduced. These two nodes are connected by an edge, which is also generated based on the template information and encapsulates the behavioral semantics represented by the template.

To ensure the coherence and integrity of information flow, after generating the new nodes and edges, the algorithm deletes the original nodes and edges and reconnects the nodes outside the reduced region to the application node and the new node. The reconnection follows two rules to maintain the unaffected information flow outside the reduced region:

- If a node outside the region is a predecessor of a node within the region, it is reconnected to the application node, becoming its predecessor.

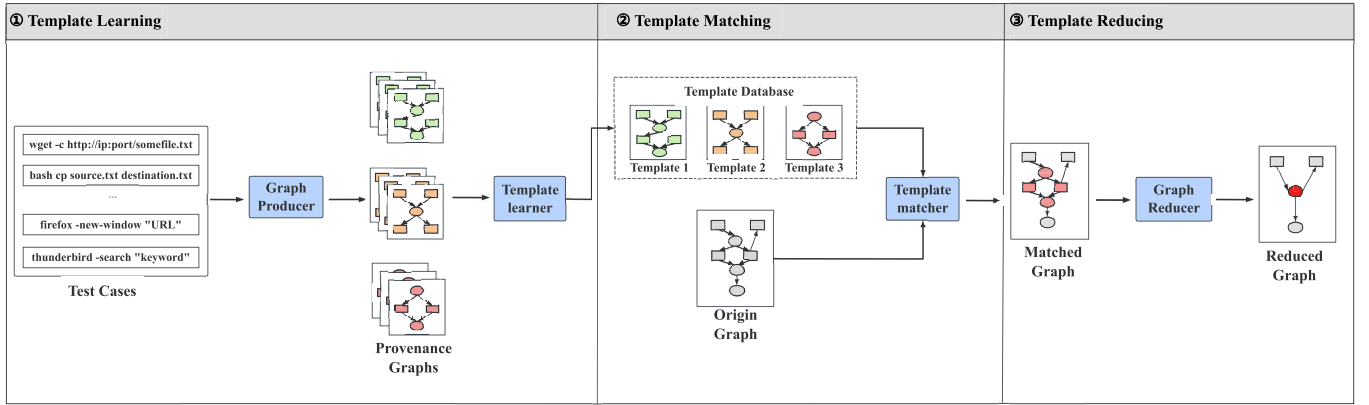


Fig. 3. The architecture of TeRed. In the template learning stage, the method extracts a large number of template subgraphs using test cases. In the template matching stage, the method attempts to find isomorphic subgraph portions in the provenance graph to be reduced that correspond to the template subgraphs. In the reducing phase, the method reduces the matched portions.

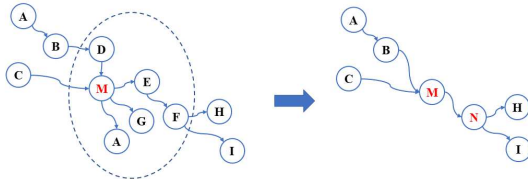


Fig. 4. Single template reduction.

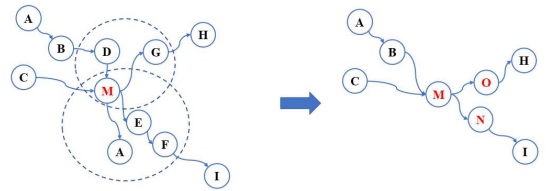


Fig. 5. Multiple template reduction (same application).

- If a node outside the region is a successor of a node within the region, it is reconnected to the new node, becoming its successor.

By adhering to these reconnection rules, the reduced information flow remains consistent with the origin information flow, thus preserving the causal relationships of the external nodes.

As illustrated in Figure 4, the region to be reduced is reduced to two nodes and one edge, with the original external associations retained. In the figure, node M represents the application node corresponding to the template, and node N is the new node introduced during reduction. The edge connecting them is also newly created.

In practical applications, a provenance graph may contain multiple normal behaviors, resulting in the presence of multiple regions to be reduced. Based on the aforementioned reduction method, the reduction process may encounter three scenarios.

1) *Non-Overlapping Regions*: This is the simplest scenario. Each independent region can be reduced separately according to the previously described method. Figure 2 provides a simple example.

2) *Overlapping Regions With the Same Application*: If two template subgraphs overlap and their corresponding applications are the same, the algorithm preserves the application node while adding two new nodes. The subsequent steps are similar to the single-template approach: the original predecessor nodes are connected to the application node, and the original successor nodes are connected to the two new nodes, as illustrated in Figure 5.

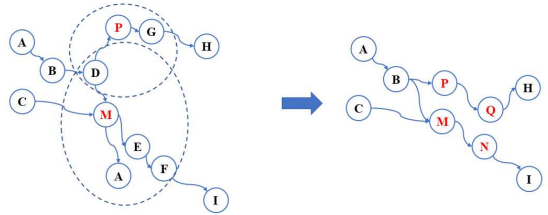


Fig. 6. Multiple template reduction (different applications).

3) *Overlapping Regions With Different Applications*: When two template subgraphs overlap and their corresponding applications differ, both application nodes are retained, and two new nodes are added. The original predecessor nodes are connected to their respective application nodes, while the original successor nodes are connected to the two new nodes. This process is depicted in Figure 6. In Figure 7, we present a practical example of provenance graph reduction, where certain nodes in the graph have been simplified for clarity.

This template-based reduction method preserves critical information within the information flow, thus ensuring a reliable data foundation for subsequent intrusion detection and attack investigation while completing the reduction process. This method not only effectively reduces the complexity of the graph but also maintains the dependency relationships between external nodes and those within the reduced regions, fundamentally facilitating efficient causal analysis. This robust support is pivotal for handling large-scale provenance graphs and real-time intrusion detection, enhancing the efficiency and accuracy of the analysis process.

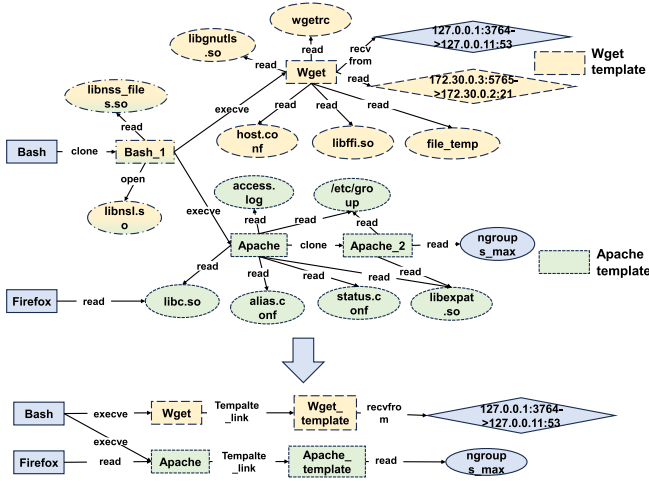


Fig. 7. Example of provenance graph reduction: The figure illustrates overlapping regions corresponding to different template matches for Wget and Apache. The reduction algorithm consolidates these two regions into four nodes and connects them to the nodes outside the regions.

V. IMPLEMENTATION

We implemented a prototype system of TeRed in Python, which is divided into a template learning module and a reduction module. Both the learning and reduction modules accept a provenance graph as input. The attribute information of nodes in the provenance graph must include at least a unique identifier (id) and an entity type (type, such as file or process). The attribute information of edges in the provenance graph must include at least a timestamp, event type, and the ids of the source and target nodes. Additionally, we implemented a provenance graph construction module that can accept sysdig logs as input and output a provenance graph that meets the above requirements. The system's output is a reduced provenance graph, which can be saved to a Neo4j graph database to support further visualized provenance analysis.

To effectively evaluate TeRed, we reproduced a set of representative methods, including CPR [20], PCAR [20], FDPR [5], SDPR [5], LogApprox [21], LogGC [18] and NodeMerge [19]. In implementing these methods, we endeavored to replicate the techniques described in their respective papers as accurately as possible, and we referred to the implementation of FAuST [28]. It is important to note that our reproductions of these works are centered around the provenance graph, so there may be deviations from the original systems. Additionally, we used three intrusion detection algorithms: Unicorn [2], ProvDetector [7], and DeepLog [6]. For Unicorn and DeepLog, we used the open-source algorithms from the original papers, while ProvDetector was reproduced based on the descriptions in the paper.

VI. EVALUATION

We evaluated TeRed to answer the following research questions (RQ):

RQ1: Can the template subgraph extracted by TeRed for a specific normal behavior accurately represent the behavior pattern corresponding to that behavior? (VI-B)

TABLE I
LAB DATASET

Dataset	Scenarios	# of Nodes	# of Edges
Lab-1	CVE-2019-9193	47278	311691
Lab-2	CVE-2016-4971	46329	132353
Lab-3	CVE-2014-6271	2570	137580

RQ2: How effective is TeRed in reducing provenance graphs? (VI-C)

RQ3: Can TeRed ensure the accuracy of subsequent attack detection and provenance? (VI-D, VI-E)

A. Dataset

We evaluated TeRed using the DARPA Transparent Computing program dataset. We selected the Trace and Theia parts of Engagement 3, which were generated during the red team and blue team adversarial engagement in April 2018. These parts contain events from a series of hosts and ground truth information about the attacks. Many previous works have used this data to evaluate their systems. We parsed the logs in the dataset and constructed provenance graphs that meet the format requirements for evaluation.

In addition, we constructed three supplementary scenarios. These three scenarios are based on three popular publicly known vulnerabilities from Vulhub: CVE-2019-9193, CVE-2016-4971, and CVE-2014-6271. CVE-2019-9193 is a vulnerability in PostgreSQL that affects multiple versions from 9.3 to 11.2, allowing some database users to execute arbitrary code via PostgreSQL. CVE-2016-4971 is a vulnerability in versions of Wget prior to 1.18, where a remote attacker can exploit this vulnerability to write arbitrary files by handling HTTP service redirections. CVE-2014-6271 is a well-known Bash vulnerability, where in versions prior to 4.3, an attacker can exploit this vulnerability to execute arbitrary scripts by crafting the values of environment variables. These vulnerabilities illustrate attacker behavior in real-world scenarios from multiple perspectives. We constructed scenarios with these vulnerabilities in a Linux environment, executed the attack actions, and collected the corresponding data, forming three corresponding datasets, which we refer to as Lab 1, Lab 2, and Lab 3. The information about the three datasets is shown in Table I, and the detailed procedures for executing the attacks are provided in Appendix A (see Supplementary Material).

B. Template Extraction

Our first research question is whether the templates extracted by TeRed during the learning phase faithfully capture the targeted normal behavior patterns. During the learning phase, to ensure that the extracted templates can accurately represent a behavior, the system executes the same test case multiple times and uses the data obtained from multiple executions for template mining. However, we do not know exactly how many times it needs to be executed. Therefore, to verify the effectiveness of the extraction method and determine the appropriate number of executions, we conducted experiments.

TABLE II
RESULTS OF TEMPLATE EXTRACTION EXPERIMENT

Application	# of Test Cases	# of Accurate Templates				
		n=2	n=4	n=6	n=8	n=10
Wget	33	0	14	30	33	33
Bash	73	0	41	72	73	73
Nginx	289	2	178	263	289	289

In the experiments, we first executed the test case corresponding to the same behavior n times and generated the corresponding provenance graphs. Then, we performed template mining on the obtained provenance graphs to get the template subgraph corresponding to that behavior. Pilot experiments showed that most templates stabilize by the seventh replay; we therefore executed the behavior 5 additional times ($7-2=5$) and generated the corresponding provenance graphs. Finally, we checked whether the extracted template was a subgraph of all 5 provenance graphs. If it was, we considered the extracted template to accurately represent the behavior pattern of that behavior.

In this way, we can verify the effectiveness of the template extraction method and determine the appropriate number of test case executions. We conducted this experiment on three application: Wget, Bash, and Nginx. These three programs encompass the common usage patterns observed in real-world systems, enabling us to assess the proposed method across heterogeneous behaviors. The experimental results are shown in Table II.

From the results, it can be seen that for all test cases, we were able to extract templates that accurately represent the corresponding behaviors. It can also be noted that as the number of test case executions increases, the extracted templates become more accurate. When the number of executions reaches 8, the behavior corresponding to all test cases is accurately extracted as the corresponding template subgraph. This indicates that our strategy of increasing the number of test case executions to eliminate noise is effective. Based on this conclusion, TeRed selects $n=10$ during the template extraction process to achieve accurate template extracting.

C. Reduction Performance

To evaluate the reduction effectiveness of TeRed, we ran TeRed and other methods on all datasets and calculated the reduction ratio. The results are shown in the table. Although the obtained reduction ratios differ from those in previous works, the relative reduction ratios between different methods remain largely consistent with previous findings. The performance differences are likely due to variations in the construction of provenance graphs and data statistics methods.

The experimental results are shown in Table III. SDPR exhibited the best reduction performance on four out of the five datasets, while TeRed achieved the best reduction effect on the Lab 2 dataset. In most datasets, TeRed outperformed most other methods, which fully demonstrates that TeRed is an efficient reduction technique. We speculate that TeRed did not achieve the best effect in all scenarios possibly because there are some behaviors that have not yet been learned.

TABLE III
REDUCTION RATIO(%) OF DATA REDUCTION METHODS

	Lab 1	Lab 2	Lab 3	THEIA	TRACE
LogGC	1.43	3	0	1.89	2.9
NodeMerge	0	16.94	0.08	1.06	5.6
CPR	51.04	0	97.29	13.77	19.18
PCAR	66.26	0	97.37	24.37	27.58
LogApprox	51.1	15.2	97.3	77.41	27.61
FDPR	69.56	0	97.69	90.72	46.02
SDPR	74.52	8.66	97.77	92.98	69.52
TeRed	52.02	56.46	88.26	78.7	49.9

These behaviors might originate from system activities or from programs for which we have not extracted the corresponding behavior templates.

It is also noteworthy that the TeRed method exhibits stable reduction performance across all environments. This shows that it can maintain high reduction efficiency under various circumstances. In contrast, the reduction ratios of other methods fluctuated significantly across different environments. For example, LogGC and NodeMerge had reduction ratios as low as 0 in some environments, which might indicate that these methods are ineffective in certain specific scenarios. While methods like CPR, PCAR, LogApprox, FDPR, and SDPR can achieve high reduction ratios in some environments, their performance is relatively poor in others. The stability of TeRed stems from its unique reduction method, which reduces based on a series of pre-learned behaviors in the system.

D. Support for Intrusion Detection

Data reduction needs to ensure that the reduced data remains high-quality and usable. If critical information in the data is lost during the reduction process, the reduction loses its significance. To check whether the reduced data still contains critical information, an effective method is to apply the data before and after reduction to downstream tasks and compare the changes in the metrics of these tasks to determine whether the critical information has been preserved. However, as mentioned earlier, most current reduction methods struggle to completely retain critical information. Therefore, this section will analyze the impact of reduction methods on intrusion detection algorithms through experiments to demonstrate that TeRed's reduction does not negatively affect subsequent intrusion detection. The three intrusion detection methods used are DeepLog [6], Unicorn [2], and ProvDetector [7], all of which are typical intrusion detection systems. Consistent with previous studies, we adopt the F1-score as the performance metric for the intrusion-detection algorithms. Because it simultaneously accounts for both precision and recall, the F1-score provides a comprehensive assessment of the model's effectiveness. In the experiments, we adjusted the parameters of the algorithms to achieve the best possible detection performance. The experimental results are shown in Table IV.

The experimental results show that the TeRed reduction method did not negatively impact the metrics of the three intrusion detection algorithms across the five datasets, demon-

TABLE IV
DETECTION RESULTS (F1-SCORE) OF 3 INTRUSION DETECTION SYSTEMS.(U=UNICORN, P=PROVDETECTOR, D=DEEPLUG)

		Origin	LogGC	NodeMerge	CPR	PCAR	LogApprox	FDPR	SDPR	TeRed
Lab 1	Reduction	0%	1.43%	0%	51.04%	66.26%	51.1%	69.56%	74.52%	52.02%
	U	0.66	0.66	0.66	0.59	0.7	0.61	0.6	0.6	0.75
	P	0.78	0.78	0.78	0.67	0.69	0.63	0.57	0.55	0.78
	D	0.82	0.81	0.82	0.79	0.64	0.7	0.75	0.73	0.82
Lab 2	Reduction	0%	3.00%	16.94%	0%	0%	15.2%	0%	8.66%	56.46%
	U	0.83	0.83	0.8	0.83	0.83	0.76	0.83	0.74	0.83
	P	0.62	0.62	0.6	0.62	0.62	0.6	0.62	0.53	0.65
	D	0.61	0.61	0.59	0.61	0.61	0.54	0.61	0.51	0.61
Lab 3	Reduction	0%	0%	0.08%	97.29%	97.37%	97.3%	97.69%	97.77%	88.26%
	U	0.66	0.66	0.66	0.56	0.56	0.55	0.49	0.49	0.66
	P	0.68	0.68	0.68	0.54	0.54	0.7	0.52	0.52	0.68
	D	0.74	0.74	0.74	0.72	0.72	0.64	0.62	0.62	0.74
THEIA	Reduction	0%	1.89%	1.06%	13.77%	24.37%	77.41%	90.72%	92.98%	78.7%
	U	0.85	0.85	0.85	0.8	0.74	0.55	0.48	0.44	0.85
	P	0.77	0.77	0.77	0.75	0.69	0.58	0.52	0.5	0.77
	D	0.88	0.88	0.88	0.86	0.85	0.61	0.45	0.43	0.88
TRACE	Reduction	0%	2.9%	5.6%	19.18%	27.58%	27.61%	46.02%	69.52%	49.9%
	U	0.79	0.79	0.79	0.75	0.61	0.58	0.49	0.43	0.79
	P	0.67	0.67	0.67	0.67	0.6	0.57	0.47	0.44	0.67
	D	0.58	0.58	0.58	0.51	0.51	0.48	0.4	0.37	0.58

strating significant stability. This indicates that the TeRed reduction process has minimal impact on the critical information of the original data and can effectively preserve the characteristics of the original data. Notably, in some cases, the algorithms using the reduction data achieved better results because the reduction algorithm helped the intrusion algorithms eliminate some normal suspicious paths, thereby increasing the exposure of attack behaviors. However, other methods failed to avoid negative impacts on the intrusion detection process while maintaining high reduction ratios. Although the LogGC and NodeMerge algorithms did not reduce the metrics of the intrusion detection algorithms in most cases, their reduction ratios were significantly lower than other methods. The CPR and PCAR algorithms, while effectively reducing data, had some negative impacts on the intrusion detection algorithms. The LogApprox, SDPR, and FDPR algorithms achieved high reduction ratios but at a significant cost, greatly reducing the metrics of the intrusion detection algorithms.

The experiments indicate that TeRed is the only method that achieves high reduction ratios without affecting subsequent intrusion detection tasks. It reduces only events related to normal behavior without interfering with events related to suspicious abnormal behavior, thus having the advantage of not negatively impacting intrusion detection algorithms. This is the core advantage of TeRed.

E. Support for Attack Investigation

Data reduction must ensure the integrity of dependencies between entities to support forward and backward tracing tasks. To evaluate TeRed's ability to support attack investigation, we conducted experiments on Lab1, Lab2, and Lab3. Through a simulated tracing process, we compared the preservation of dependency relationships between attack-related entities before and after reduction. Specifically, we

TABLE V
TRACING RESULTS OF 3 DATASETS.(PRE*=PRECURSOR NODES, SUCC*=SUCCESSOR NODES)

Dataset	Attack Entities	Before Reduction		After Reduction	
		pre*	succ*	pre*	succ*
Lab1	35	10	24	10	24
Lab2	68	19	48	19	48
Lab3	26	18	7	18	7

first selected a set of entities related to the attack (i.e., the attack entity set) from the origin provenance graph, and then chose one entity as the starting node (representing the Point of Interest, POI, in tracing). Through forward and backward tracing, we divided the nodes in the entity set into predecessors and successors of the starting node. Finally, we performed the same tracing operation on the same starting node in the reduced provenance graph and compared the results of the two divisions. If the results are completely consistent (i.e., no missing or incorrectly divided predecessor or successor nodes after reduction), it indicates that TeRed has not disrupted the critical entity dependencies, ensuring the reachability between nodes. The experimental results are shown in Table V.

In the experiments, the predecessor and successor nodes on the three datasets did not change before and after reduction, indicating that the tracing results remained unchanged. This demonstrates that TeRed's reduction did not disrupt the entity dependencies on the attack-related paths. By incorporating the results from the intrusion detection experiments in VI-D, we demonstrate that analysts can identify POI events in the reduced data and subsequently reconstruct the complete attack story from these POI events.

Moreover, we acknowledge that while the reduced graph may introduce slight gaps in reconstructing the attack path if it traverses the reduced areas, the preservation of essential attack events enables analysts to effectively trace the progression of

the attack. Consequently, although there are inherent trade-offs between reduction and utility—particularly concerning potential minor losses in detail—the overall capacity to reconstruct the attack narrative remains strong.

Therefore, we contend that our findings adequately address the concerns regarding the balance between data reduction and the utility of the reduced provenance graph in attack investigations.

VII. DISCUSSION

A. Scalability of the Method

The complexity and variability of modern computer systems present challenges for TeRed. Firstly, different operating environments, such as various operating systems and hardware configurations, can alter program behaviors. Secondly, program updates and feature modifications can change behavior patterns.

TeRed can adapt to changes by continuously expanding and updating its template library through new test cases. When new behavior patterns are discovered or programs are updated, we write and run new unit tests to extract and add these patterns to the library. Since our template library is based on unit test cases, as long as we continually write and run new test cases, we can continuously update and expand our template library. This allows our reduction method to continuously adapt to system changes, thereby maintaining its effectiveness and accuracy.

To demonstrate TeRed's capability, we conducted experiments. Our experiments simulated a scenario where new software is installed in the system, and TeRed's template library does not include templates related to this software. Therefore, it needs to learn the behavior patterns of the software from scratch and continuously expand the template library to improve reduction effectiveness. In the experiments, we used Lab 1-3. For each dataset, we used template files in varying proportions from 10% to 100%, increasing the proportion by 10% each time. To minimize the interference caused by some high-frequency template files, each experiment randomly selected templates ten times to conduct ten sub-experiments, and the average of the ten results was taken as the final result. Details of the software used and their corresponding test cases are provided in Appendix B (see Supplementary Material).

The experimental results are shown in the Table VI. As expected, the reduction ratio is positively correlated with the proportion of template files used. As the number of templates increases, the reduction ratio also continuously improves. The experiments demonstrate that in a system undergoing changes, TeRed can adapt to these changes by expanding its test cases. Furthermore, the experiments indicate that by increasing the number of relevant templates, more normal behaviors can be identified on the provenance graph, thereby improving TeRed's reduction ratio. This achieves the "more is better" effect.

B. Reversible Reduction

During the reduction process, TeRed removes most of the information related to matched nodes and edges. Although this information pertains to normal system behavior and might still

TABLE VI
REDUCTION RATIO CHANGES WHEN THE NUMBER
OF TEMPLATES INCREASES

Proportion of templates used	Reduction ratio
10%	14.6%
20%	26.7%
30%	37.3%
40%	47.4%
50%	52.8%
60%	55.7%
70%	58.2%
80%	60.5%
90%	61.1%
100%	62.5%

be useful for attack detection and investigation, this does not mean that TeRed cannot achieve lossless compression. Unlike other lossy reduction methods, TeRed's reduction is reversible. It can restore the deleted information using templates, thereby preventing information loss due to reduction.

Since the templates record key information about the reduced regions, we can use them to restore the deleted nodes and edges in the provenance graph. The restoration process involves two parts: restoring the attributes of nodes and edges, and restoring the original dependency relationships. For attribute restoration, the templates save the entity types of all nodes and the system call attributes of edges, but not other attributes. Therefore, an additional table is needed to store these other attributes for each node and edge. During restoration, the templates and this attribute table can be used to add the corresponding attribute values to the nodes and edges. For dependency relationships, the templates save the internal dependencies within the compressed region. However, an additional table is needed to record the original dependencies between nodes inside the compressed region and those outside of it. This table records the associations between each node inside the compressed region and the external nodes, while the templates record the associations within the compressed region. By combining the two, the original dependency relationships can be restored. Attribute information occupies significant storage space as it includes all remaining attributes of the nodes and edges. In contrast, dependency relationship information, determined by the number of edges in the graph, typically requires less space. For example, in Lab 1, the storage space required to restore the attribute information and the dependency relationship information accounts for 27.71% and 0.56% of the original dataset size. The newly introduced data structures will occupy additional storage space, but some general compression algorithms can be used to reduce their space cost.

Notably, restoration can be performed locally, allowing for the restoration of only the parts of interest without needing to restore the entire provenance graph. This flexibility makes our method adaptable to various scenarios. For instance, during attack detection, the reduced provenance graph improves detection speed. During investigation, restoring compressed regions adjacent to the attack path enhances efficiency and provides more comprehensive attack-related information. In

any case, the restored provenance graph can be directly used for attack detection and investigation.

C. Imperfect Data

Although we assume that the provenance graph is complete and untampered, real-world scenarios often lead to imperfect data due to issues such as log loss. Since TeRed targets only normal behaviors through templates, encountering imperfect data may result in some templates becoming ineffective. This could lead to a decrease in the reduction ratio. However, it is important to emphasize that such circumstances will not adversely impact the performance of attack detection and provenance algorithms.

Specifically, imperfect data can arise during both the template learning and reduction stages. In the template learning stage, sporadic data loss does not affect the template generation process, as our method executes multiple test cases to eliminate randomness, as described in VI-B. However, if there are overarching data issues, such as incorrect collector settings, erroneous templates may be extracted, leading to template matching failures and affecting the reduction ratio. During the reduction stage, data loss may result in template matching failures, preventing the reduction of the corresponding areas. This could lead to a decrease in the reduction ratio, but since the original data remains intact, attack detection and investigation will not be compromised.

D. Integration With Existing Work

FAuST's research has demonstrated that combining multiple existing reduction techniques can achieve better reduction results [28]. This is because different reduction methods may focus on and optimize different aspects, and by using them together, their respective advantages can be fully utilized while compensating for each other's shortcomings, thereby achieving better reduction performance.

On one hand, TeRed, as a reduction method that directly operates on provenance graphs, takes standard-format provenance graphs as both input and output, which gives it excellent generality and compatibility. On the other hand, TeRed only reduces regions that match the templates, allowing other compression methods to be applied to further reduce the remaining regions. Therefore, TeRed can be easily combined with other reduction methods suitable for provenance graphs, achieving better overall reduction results.

TeRed can also be combined with other graph compression methods to further reduce data storage pressure. The work by Fei et al. [29] achieved lossless compression of provenance graphs and supported real-time historical event retrieval. Combining this with TeRed can further reduce storage costs while improving retrieval efficiency. Additionally, there are numerous graph compression algorithms widely applied in various graph analysis-related fields (such as social networks, machine learning, and data mining) [30], [31], [32], [33], [34], [35], [36], [37], [38], [39]. TeRed can also be combined with these algorithms to further enhance space compression rates. However, this is not the focus of this paper, so it will not be elaborated further.

VIII. RELATED WORK

A. Lossless Compression Methods

Lossless Compression techniques ensure that no information is lost during the optimization process, which is crucial for the integrity and accuracy of log data. Logzip [40] identifies log repetition patterns through hierarchical aggregation and then uses incremental dictionary compression to similar log entries to ensure that log data does not lose any information during compression and decompression. SEAL [29] is based on information-theoretic observations of system event data, achieving lossless compression by analyzing the structure of log entries and identifying and extracting common parts. Elise [41] is an efficient storage and logging system that supports all types of security analysis, using a new method for preprocessing log files and learning optimal character encoding through deep neural networks (DNN) for lossless log compression. CLP [42] achieves higher compression ratios and faster search performance by using domain-optimized compression and search algorithms that leverage the high repetition in text logs. LogBlock [43] improves the compression ratio of log files by preprocessing log headers and rearranging log content to reduce redundancy.

B. Lossy Reduction Methods

Lossy reduction can discard some less important data to further reduce data storage requirements and improve processing efficiency. Lossy reduction retains key information while allowing some degree of information loss. KCAL [3] uses prior knowledge to identify important parts in logs, reducing redundant events before they are generated, reducing the overhead of transmitting, writing, and storing redundant events. LogGC [18] uses garbage collection techniques to identify and delete redundant information in logs to optimize log storage efficiency. NodeMerge [19] proposes an online system event storage data reduction system based on templates, achieving data reduction based on the access pattern of read-only file events. CPR/PCAR [20] observes that some events contribute equally to dependency analysis, attempting to aggregate them to ensure high-quality forensic analysis. Traditional log-based reduction methods merge log entries with the same meaning, and FDPR [5] extends this idea to graph-based reduction methods, merging edges with the same reachability. SDPR [5] traces suspicious nodes back to the source node and then performs forward analysis that only retains the dependencies of the source node, thus achieving reduction effects. LogApprox [21] is a regex learning method that eliminates redundant file I/O related to benign process activities by learning the regex of a given process. FAuST [28] achieves efficient log reduction and the retention of key information by mixing various reduction and summarization techniques.

IX. CONCLUSION

This paper proposes TeRed, a novel method for reducing provenance graphs based on normal behavior patterns. Our method utilizes unit tests to learn the normal behavior patterns of the system and then reduces the provenance graph using the learned patterns. We conducted experiments on five datasets,

and the results show that our method effectively reduces the size of the provenance graph while preserving complete attack-related information. It does not negatively impact attack detection and investigation, demonstrating significant advantages over other data reduction methods.

REFERENCES

- [1] Z. Xu, P. Fang, C. Liu, X. Xiao, Y. Wen, and D. Meng, "DEPCOMM: Graph summarization on system audit logs for attack investigation," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 540–557.
- [2] X. Han, T. Pasquier, A. Bates, J. Mickens, and M. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [3] S. Ma et al., "Kernel-supported cost-effective audit logging for causality tracking," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 241–253.
- [4] M. A. Inam et al., "SoK: History is a vast early warning system: Auditing the provenance of system intrusions," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2620–2638.
- [5] M. N. Hossain et al., "Dependence-preserving data compaction for scalable forensic analysis," in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 1723–1740.
- [6] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [7] Q. Wang et al., "You are what you do: Hunting stealthy malware via data provenance analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–17.
- [8] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 1035–1044.
- [9] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "PROGRAPHER: An anomaly detection system based on provenance graph embedding," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 4355–4372.
- [10] X. Han et al., "SIGL: Securing software installations through deep graph learning," in *Proc. 30th USENIX Secur. Symp.*, 2020, pp. 2345–2362.
- [11] D. Han et al., "DeepAID: Interpreting and improving deep learning-based anomaly detection in security applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3197–3217.
- [12] L. Yu et al., "ALchemist: Fusing application and audit logs for precise attack provenance without instrumentation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.
- [13] R. Wang, Y. Peng, Y. Sun, X. Zhang, H. Wan, and X. Zhao, "TeSec: Accurate server-side attack investigation for Web applications," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 2799–2816.
- [14] W. U. Hassan, M. A. Nouredine, P. Datta, and A. Bates, "OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–16.
- [15] S. Ma, J. Zhai, F. Wang, K. H. Lee, X. Zhang, and D. Xu, "MPI: Multiple perspective attack investigation with semantic aware execution partitioning," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1111–1128.
- [16] Y. Kwon et al., "MCI: Modeling-based causality inference in audit logging for attack investigation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [17] S. M. Milajerdi, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "POIROT: Aligning attack behavior with kernel audit records for cyber threat hunting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, 2019, pp. 1795–1812.
- [18] K. H. Lee, X. Zhang, and D. Xu, "LogGC: Garbage collecting audit log," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 1005–1016.
- [19] Y. Tang et al., "NodeMerge: Template based efficient data reduction for big-data causality analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 1324–1337.
- [20] Z. Xu et al., "High fidelity data reduction for big data security dependency analyses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Vienna, Austria, Oct. 2016, pp. 504–516.
- [21] N. Michael, J. Mink, J. Liu, S. Gaur, W. U. Hassan, and A. Bates, "On the forensic validity of approximated audit logs," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2020, pp. 189–202.
- [22] T. Pasquier et al., "Practical whole-system provenance capture," in *Proc. Symp. Cloud Comput.*, Santa Clara, CA, USA, Sep. 2017, pp. 405–418.
- [23] A. Bates, D. Tian, R. B. K. Butler, and T. Moyer, "Trustworthy whole-system provenance for the Linux kernel," in *Proc. USENIX Conf. Secur. Symp. (SEC)*, Austin, TX, USA, Aug. 2015, pp. 319–334.
- [24] R. Paccagnella et al., "Custos: Practical tamper-evident auditing of operating systems using trusted execution," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [25] R. Paccagnella, K. Liao, D. Tian, and A. Bates, "Logging to the danger zone: Race condition attacks and defenses on system audit frameworks," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security*, Nov. 2020, pp. 1551–1574.
- [26] X. Yan and J. Han, "gSpan: Graph-based substructure pattern mining," in *Proc. IEEE Int. Conf. Data Min.*, Maebashi City, Japan, Jan. 2002, pp. 721–724.
- [27] L. P. Cordella, P. Foggia, C. Sansone, and M. Vento, "A (sub)graph isomorphism algorithm for matching large graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 10, pp. 1367–1372, Oct. 2004.
- [28] M. A. Inam et al., "FAuST: Striking a bargain between forensic auditing's security and throughput," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 813–826.
- [29] F. Peng, Z. Li, Z. Wang, X. Yu, D. Li, and K. Jee, "SEAL: Storage-efficient causality analysis on enterprise logs with query-friendly compression," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 2987–3004.
- [30] Z. Chen et al., "CompressGraph: Efficient parallel graph analytics with rule-based compression," *Proc. ACM Manage. Data*, vol. 1, no. 1, pp. 1–31, May 2023.
- [31] A. Apostolico and G. Drovandi, "Graph compression by BFS," *Algorithms*, vol. 2, no. 3, pp. 1031–1044, Aug. 2009.
- [32] P. Boldi, M. Rosa, M. Santini, and S. Vigna, "Layered label propagation: A multiresolution coordinate-free ordering for compressing social networks," in *Proc. 20th Int. Conf. World Wide Web*, 2011, pp. 587–596.
- [33] F. Chierichetti, R. Kumar, S. Lattanzi, M. Mitzenmacher, A. Panconesi, and P. Raghavan, "On compressing social networks," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jun. 2009, pp. 219–228.
- [34] N. R. Brisaboa, S. Ladra, and G. Navarro, "K2-trees for compact Web graph representation," in *Proc. 16th Int. Symp. String Process. Inf. Retr.*, 2009, pp. 18–30.
- [35] F. Claude and G. Navarro, "Fast and compact Web graph representations," *ACM Trans. Web*, vol. 4, no. 4, pp. 1–31, Sep. 2010.
- [36] J. Shun and G. E. Blelloch, "Ligra: A lightweight graph processing framework for shared memory," in *Proc. 18th ACM SIGPLAN Symp. Princ. Pract. Parallel Program. (PPoPP)*, 2013, pp. 135–146.
- [37] J. Sun, E. M. Bollt, and D. Ben-Avraham, "Graph compression—Save information by exploiting redundancy," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 6, Jun. 2008, Art. no. P06001.
- [38] W. Liu et al., "On compressing weighted time-evolving graphs," in *Proc. 21st ACM Int. Conf. Inf. Knowl. Manage.*, Oct. 2012, pp. 2319–2322.
- [39] S. Maneth and F. Peternek, "Compressing graphs by grammars," in *Proc. IEEE 32nd Int. Conf. Data Eng. (ICDE)*, May 2016, pp. 109–120.
- [40] J. Liu, J. Zhu, S. He, P. He, Z. Zheng, and M. R. Lyu, "Logzip: Extracting hidden structures via iterative clustering for log compression," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2019, pp. 863–873.
- [41] H. Ding, S. Yan, J. Zhai, and S. Ma, "ELISE: A storage efficient logging system powered by redundancy reduction and representation learning," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3023–3040.
- [42] K. Rodrigues, Y. Luo, and D. Yuan, "CLP: Efficient and scalable search on compressed text logs," in *Proc. 15th USENIX Symp. Operating Syst. Design Implement. (OSDI 21)*, 2021, pp. 183–198.
- [43] K. Yao, M. Sayagh, W. Shang, and A. E. Hassan, "Improving state-of-the-art compression techniques for log management tools," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2748–2760, Aug. 2022.